

DDA5001 Machine Learning

Quick Final Review

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Main Components

- ▶ Concepts and feasibility of Learning, perceptron.
- ▶ Linear regression, least squares, MLE.
- ▶ Training versus testing.
- ▶ Overfitting, validation, regularization.
- ▶ Logistic regression.
- ▶ Gradient-based optimization algorithms.
- ▶ SVM, kernel methods.
- ▶ Dimensionality reduction, PCA.
- ▶ Clustering, k-means.
- ▶ Neural networks, backpropagation, SGD and Adam
- ▶ Transformer, and LLMs.

You are supposed to

- ▶ know the basic concepts, idea, and insights.
- ▶ can derive simple results using learned concepts.

Supervised Learning: Components

Consider the credit approval problem.

Formalization:

- ▶ Input space: $\mathcal{X}$ (all possible inputs, e.g., customer's data)
- ▶ Output space: $\mathcal{Y}$ (All possible outputs, e.g., the labels)
- ▶ Target function $g : \mathcal{X} \rightarrow \mathcal{Y}$ (underlying credit approval model)
- ▶ Training dataset: $(x_1, y_1), \dots, (x_n, y_n)$ (historical records)
- ▶ Hypothesis/model: f_{θ} (model to be determined)
- ▶ Optimization algorithm: $\mathcal{A}$ (learning the model from data)

The Perceptron Learning Algorithm

Perceptron is used for **binary classification**.

- ▶ Given the training dataset $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$, with $y_i = \pm 1$.

- ▶ The perceptron model

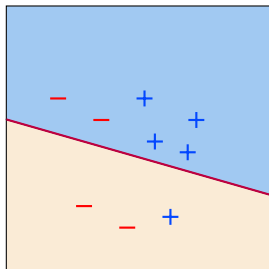
$$f_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x}$$

- ▶ Pick a **misclassified** data

$$\text{sign}(\theta^{\top} \mathbf{x}_i) \neq y_i$$

- ▶ Update rule

$$\theta = \theta + y_i \mathbf{x}_i$$



Feasibility of Learning

- ▶ The (really) unknown target function g is the object of learning.
Learning is all about to infer g outside of the seen training dataset.
- ▶ In general, unfortunately, learning is not feasible (without assumptions).
- ▶ Fortunately, if we assume i.i.d. on the training and testing data, learning will be possible in a probabilistic way. $\leadsto$ Theory of learning.

Errors and Goal

- **In-sample Error:** Given a set of **training samples** $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$,

$$\text{Er}_{\text{in}} = \frac{1}{n} \sum_{i=1}^n e(f(\mathbf{x}_i), g(\mathbf{x}_i))$$

- **Out-of-sample Error:** Suppose data $\mathbf{x}$ follows a certain distribution $\mathcal{D}$ in an i.i.d. manner,

$$\text{Er}_{\text{out}} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [e(f(\mathbf{x}), g(\mathbf{x}))]$$

- **Goal:** Make Er_{out} small.

Linear Regression and Least Squares

Data: Given past sales data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ (housing price prediction), where

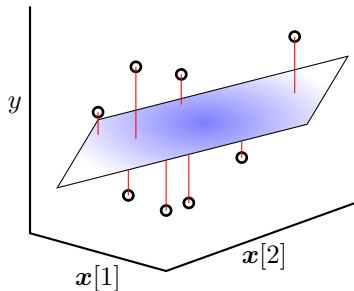
$\mathbf{x}_i \in \mathbb{R}^d$ represents square meter, metro, schools...

y_i represents price

Hypothesis/model:

$$y_i \approx f_{\boldsymbol{\theta}}(\mathbf{x}_i) = \boldsymbol{\theta}^\top \mathbf{x}_i,$$

where $\boldsymbol{\theta} \in \mathbb{R}^d$.



LS formulation:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\theta})^2 \iff$$

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Solution of Least Squares

For LS, the optimal solution $\hat{\boldsymbol{\theta}}$ satisfies

$$\mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\theta}} = \mathbf{X}^\top \mathbf{y}$$

If $\mathbf{X} \in \mathbb{R}^{n \times d}$ has full column rank (i.e., $\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$ is full rank), we have

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}.$$

If $\mathbf{X} \in \mathbb{R}^{n \times d}$ does not have full column rank, we have infinitely many solutions.

Maximum Likelihood Estimation

- ▶ Observe (i.i.d.) data $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ following $p(\mathbf{x}|\boldsymbol{\theta}^*)$.
- ▶ Build the likelihood function $p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{i=1}^n p(\mathbf{x}_i|\boldsymbol{\theta})$ for some parameter $\boldsymbol{\theta}$.
- ▶ Log-likelihood $\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \log(p(\mathbf{x}_i|\boldsymbol{\theta}))$
- ▶ Maximum likelihood estimator: $\hat{\boldsymbol{\theta}}_{MLE} = \operatorname{argmax}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$.

Key point: Know $p(\mathbf{x}|\boldsymbol{\theta})$.

- ▶ Least square is a maximum likelihood estimator under Gaussian noise assumption.

Concentration Inequalities

Theorem: Sub-Gaussian Concentration

Suppose X is a sub-Gaussian random variable with mean μ and parameter σ , then for any $t > 0$, we have

$$\Pr[|X - \mu| \geq t] \leq 2e^{-\frac{t^2}{2\sigma^2}}$$

Corollary: Hoeffding's inequality for bounded random variables

Suppose X_i are **independent** random variables with mean μ_i and bounded on $[a_i, b_i]$ for $i = 1, \dots, n$, then for any $t > 0$, we have

$$\Pr\left[\sum_{i=1}^n (X_i - \mu_i) \geq t\right] \leq e^{-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}}$$

► Concentration is the fundamental of learning theory.

Generalization Result for Finite Hypothesis Space

Theorem: Generalization bound — **finite hypothesis space**

Let $\mathcal{H}$ be a **finite** model space. For any $\delta > 0$, the following generalization bound holds **with probability at least $1 - \delta$**

$$\forall f \in \mathcal{H} \quad \text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \sqrt{\frac{\log\left(\frac{2|\mathcal{H}|}{\delta}\right)}{2n}} \quad (1)$$

- ▶ The dependence on δ is only **logarithmically**.
- ▶ The generalization error increases when $|\mathcal{H}|$ grows, but only **logarithmically** (only for finite hypothesis space case).
- ▶ **More samples (larger n) lead to better generalization (universal).**
- ▶ Not practical due to finite hypothesis assumption.

Dichotomy, Growth Function, VC Dimension

Dichotomies of $\mathcal{H}$

Let $\{x_1, \dots, x_n\}$. The dichotomies generated by $\mathcal{H}$ on these points are defined by

$$\mathcal{H}(x_1, \dots, x_n) = \{(f(x_1), \dots, f(x_n)) : f \in \mathcal{H}\}.$$

Growth function

The growth function for the hypothesis set $\mathcal{H}$ is defined as:

$$\mathcal{G}_{\mathcal{H}}(n) = \max_{\{x_1, \dots, x_n\} \subseteq \mathcal{X}} |\mathcal{H}(x_1, \dots, x_n)|.$$

VC dimension

The VC dimension of a hypothesis space $\mathcal{H}$, denoted by $d_{\text{VC}}(\mathcal{H})$ or simply d_{VC} , is the largest n so that it can be shattered by $\mathcal{H}$, i.e.,

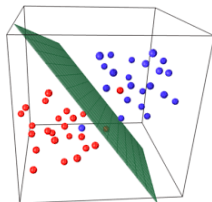
$$d_{\text{VC}}(\mathcal{H}) := \max\{n : \mathcal{G}_{\mathcal{H}}(n) = 2^n\}.$$

If $\mathcal{G}_{\mathcal{H}}(n) = 2^n$ for all n , i.e., no break point for $\mathcal{H}$, then $d_{\text{VC}}(\mathcal{H}) = \infty$.

VC Dimension of Linear Classifier

When $d = 2$, we have $d_{VC} = 3$ for (binary) linear classifier such as perceptron.

This can be generalized to the following general result:



Theorem

For d -dimensional (binary) linear classifier, we have

$$d_{VC} = d + 1.$$

VC Dimension Generalization Result

VC generalization result

For any $\delta > 0$, with probability at least $1 - \delta$, we have the following generalization bound:

$$\forall f \in \mathcal{H} \quad \text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \sqrt{\frac{8}{n} \log \left(\frac{4\mathcal{G}_{\mathcal{H}}(2n)}{\delta} \right)}$$

Upon invoking the upper bound on growth function using VC dimension, we have

$$\forall f \in \mathcal{H} \quad \text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \sqrt{\frac{8}{n} \log \left(\frac{4((2n)^{d_{\text{vc}}} + 1)}{\delta} \right)}$$

The Fundamental Trade-off

$$\forall f \in \mathcal{H} \quad \text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \mathcal{O} \left(\sqrt{\frac{d_{\text{VC}}}{n}} \right)$$

To make Er_{out} small:

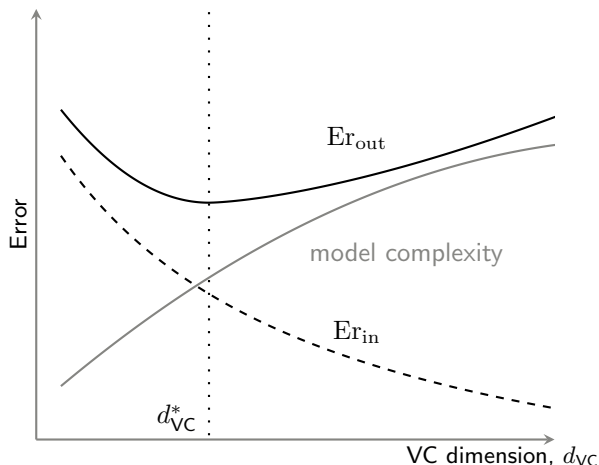
On the **training** side, we need:

more complex hypothesis $\mathcal{H}$ (larger d_{VC})

On the **generalization** side, we need:

less complex hypothesis $\mathcal{H}$ (smaller d_{VC})

Learning Curve from VC Analysis



- ▶ The **optimal model** is the one that minimizes the combinations of Er_{in} and generalization error.
- ▶ Occam's Razor principle: **The simplest workable model is the best.**

Logistic Regression Model for Binary Classification

Logistic regression model:

$$\Pr[y|\mathbf{x}; \boldsymbol{\theta}] = \frac{1}{1 + \exp\left(-y \cdot \boldsymbol{\theta}^\top \mathbf{x}\right)}$$

Maximum likelihood estimation leads to the learning problem:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \underbrace{\log\left(1 + \exp\left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i\right)\right)}_{e(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)}$$

Multi-class LR:

$$\hat{\boldsymbol{\Theta}} = \underset{\boldsymbol{\Theta} \in \mathbb{R}^{d \times K}}{\operatorname{argmin}} -\frac{1}{n} \sum_{i=1}^n \sum_{\ell=1}^K 1_{\{y_i=\ell\}} \log\left(\frac{\exp(\boldsymbol{\theta}_\ell^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\boldsymbol{\theta}_j^\top \mathbf{x}_i)}\right),$$

Gradient-based Learning Algorithm

Consider the abstract learning problem:

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}).$$

Gradient descent (GD)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

GD with momentum:

$$\begin{aligned}\boldsymbol{\theta}_{k+1} &= \boldsymbol{\theta}_k - \boldsymbol{m}_k \\ \boldsymbol{m}_k &= \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k) + \beta_k \boldsymbol{m}_{k-1}\end{aligned}$$

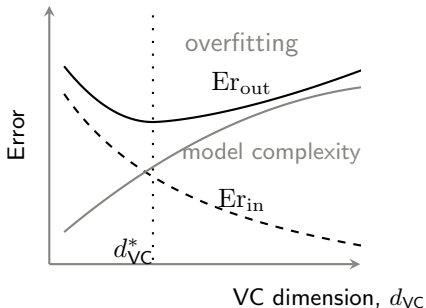
GD with Nesterov's momentum:

$$\begin{aligned}\boldsymbol{\theta}_{k+1} &= \boldsymbol{w}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{w}_k) \\ \boldsymbol{w}_k &= \boldsymbol{\theta}_k + \frac{k-1}{k+2}(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1})\end{aligned}$$

Overfitting

Overfitting

Fitting the data more than is needed



- ▶ One possible reason: $\mathcal{H}$ is more complex than is needed.
- ▶ It means fitting the observed data well no longer indicates that we will get a small out-of-sample error, and may actually lead to the opposite effect. That is, overfitting leads to small training error but bad generalization (large Er_{out}).

Overfitting: Catalysts

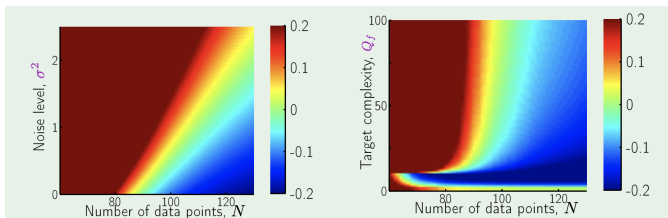


Figure: Color means the relative value of Er_{out} (i.e., the overfitting level).

Overfitting: Catalysts

- ▶ **Number of training samples** increase, overfitting decreases.
- ▶ **Noise** in data increase, overfitting increases.
- ▶ **Target model complexity** increases, overfitting increases.

Validation as A Cure for Overfitting

Validation technique tries to estimate the out-of-sample error for eliminating overfitting:

$$\underbrace{\text{Er}_{\text{out}}(f)}_{\text{validation estimates this quantity}} \leq \text{Er}_{\text{in}}(f) + \text{overfit penalty}.$$

Validation error:

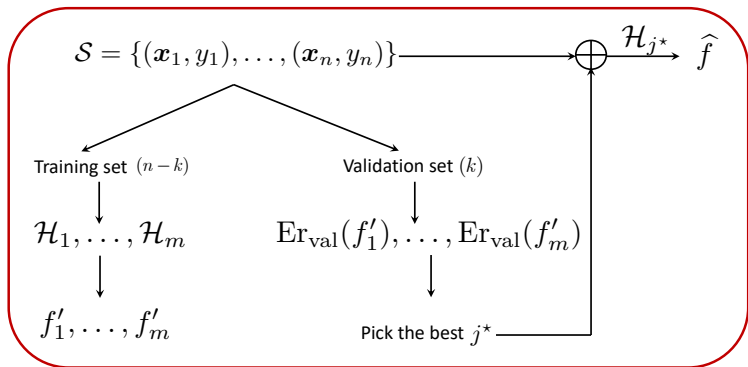
$$\text{Er}_{\text{val}}(f') = \frac{1}{k} \sum_{i=1}^k e(f'(\mathbf{x}_i), y_i)$$

We have

$$\text{Er}_{\text{out}}(f') \leq \text{Er}_{\text{val}}(f') + \mathcal{O}\left(\frac{1}{\sqrt{k}}\right).$$

Hence, we can use validate set to check whether we have a good or bad learned model, **model selection**.

Validation for Model Selection

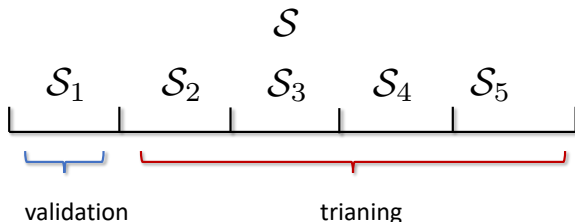


Cross Validation

- **k -fold cross validation**: Choose a batch of data points for validation rather than one point (as in the Leave One Out method).

In k -fold cross validation, k is the number of folds and $k' = \frac{n}{k}$ is the size of the validation set

Example: $k = 5$



Common choice: $k = 5, 10$.

Regularization is Another Cure for Overfitting

Regularization is another weapon for eliminating **overfitting**, which **penalizes** the model complexity using **penalty** $\Omega(f)$:

$$\min_{f \in \mathcal{H}} \text{Er}_{\text{in}}(f) + \Omega(f).$$

Typical regularizers:

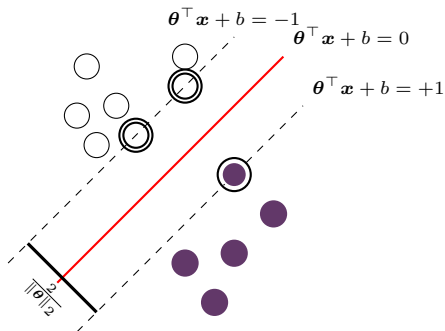
- ▶ ℓ_2 -regularization / weight decay

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2.$$

- ▶ ℓ_1 -regularization

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^d |\theta_i|,$$

Hard-margin SVM



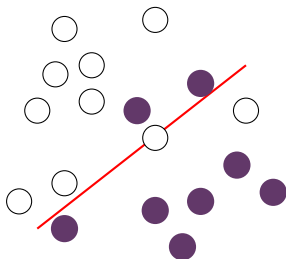
Hard-margin SVM:

$$\underset{\theta \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \|\theta\|_2^2$$

$$\text{subject to } y_i(\theta^\top x_i + b) \geq 1, \quad i = 1, \dots, n$$

Hard-margin SVM can be regarded as: Choose a classifier from all possible perceptron classifiers that has the largest margin.

Soft-margin SVM



Soft-margin SVM:

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)) + \lambda \|\boldsymbol{\theta}\|_2^2.$$

- ▶ $h(\boldsymbol{\theta}; \mathbf{x}_i, y_i) = \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b))$ is the **hinge-loss**.
- ▶ Soft-margin SVM strictly generalizes the the hard-margin SVM to **non-linearly separable** case.
- ▶ We can apply the subgradient method for solving it.

Kernel Method / Trick

- ▶ After applying the nonlinear transform Φ , the inner product becomes

$$\Phi(\mathbf{x})^\top \Phi(\mathbf{x}').$$

- ▶ Kernel method is a trick to evaluate the above inner product **without explicitly calculating $\Phi(\mathbf{x})$ and $\Phi(\mathbf{x}')$** .

Kernel

A function $\kappa : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a **kernel** for a nonlinear transform Φ if

$$\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\top \Phi(\mathbf{x}'), \quad \forall \mathbf{x}, \mathbf{x}'$$

Typical kernels:

- ▶ The polynomial kernel of degree m :

$$\kappa(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^m.$$

- ▶ Gaussian / Radial basis function (RBF) kernel:

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{2\sigma^2}\right)$$

The PCA

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

- ▶ Remove the mean

$$\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu},$$

where $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.

- ▶ Form the empirical covariance matrix from data

$$\mathbf{S} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X} \mathbf{X}^\top.$$

- ▶ Compute the eigen decomposition of $\mathbf{S}$

$$\mathbf{S} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top$$

and the PCA solution is given by the first k columns of $\mathbf{U}$:

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k].$$

- ▶ Compute principle component and low-dimensional sample

$$\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i, \quad \mathbf{A} \boldsymbol{\theta}_i = \mathbf{A} \mathbf{A}^\top \mathbf{x}_i.$$

k-means

k-means criterion: Choosing C to **minimize**

$$\min_C W(C) = \sum_{j=1}^k \sum_{i: C(i)=j} \|x_i - \mu_j\|_2^2$$

where

$$\mu_j := \frac{1}{n_j} \sum_{i: C(i)=j} x_i \quad n_j = |\{i : C(i) = j\}|$$

k-means algorithm

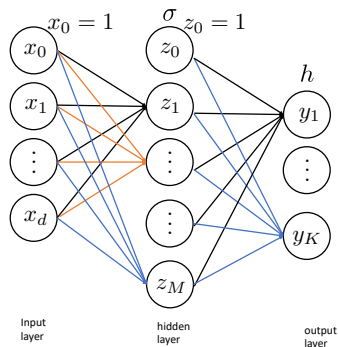
► Fix $C = C^{(t)}$, the update of $\{m_j\}_{j=1}^k$ is

$$m_j^{(t+1)} = \frac{1}{n_j} \sum_{i: C^{(t)}(i)=j} x_i, \quad j = 1, \dots, k.$$

► Fix $\{m_j\}_{j=1}^k = \{m_j^{(t+1)}\}_{j=1}^k$, the update of C is

$$C^{(t+1)}(i) = \operatorname{argmin}_{1 \leq j \leq k} \|x_i - m_j^{(t+1)}\|_2^2, \quad i = 1, \dots, n.$$

One Hidden Layer (Two-Layer) Neural Network



Use $\mathbf{V}$ and $\mathbf{W}$ to denote the weight matrices of the first layer and second layer. NN model can be written as

$$\mathbf{y} = f_{\theta}(\mathbf{x}) = h(\mathbf{W}\sigma(\mathbf{V}\mathbf{x})).$$

- The input of the next layer is the output of the previous layer ($\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$).

Learning Formulation and BP

- ▶ NN training is the same as linear regression or classification learning problem, with the difference being using the **learned** $\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$ as input rather than the sample $\mathbf{x}$. E.g., binary classification:

$$\min_{\boldsymbol{\theta}=(\mathbf{w}, \mathbf{V})} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \log (1 + \exp (-y_i \cdot \mathbf{w}^\top \sigma(\mathbf{V} \mathbf{x}_i))) \right\}.$$

- ▶ In order to train NN, we need to compute the gradient of $\mathcal{L}$.
 $\rightsquigarrow$ **backpropagation (BP)**, which is just **chain rule**.
- ▶ BP has forward pass (compute network value) and backward pass (compute gradient).

SGD and Adam

Stochastic gradient descent: SGD

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k).$$

Adam

1. Compute a stochastic gradient $\mathbf{g}_k = \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$;
2. Update momentum: $\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k$;
3. Update adaptive learning rate $\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2$;
4. Bias-corrected momentum: $\mathbf{m}_k = \mathbf{m}_k / (1 - \beta_1^k)$;
5. Bias-corrected adaptive learning rate: $\mathbf{v}_k = \mathbf{v}_k / (1 - \beta_2^k)$;
6. Update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \frac{\mathbf{m}_k}{\sqrt{\mathbf{v}_k + 10^{-8}}}.$$

Repeat until convergence.

- We often use **AdamW** to have the correct regularization.

Deep Learning

Deep learning is a direct generalization of the former two-layer neural network model.

- Deep feed-forward networks:

$$f_{\theta}(x) = h(\mathbf{W}^L \sigma(\mathbf{W}^{L-1} \dots \sigma(\mathbf{W}^2 \sigma(\mathbf{W}^1 x))))$$

Training: BP and Adam method is the same as two-layer NN.

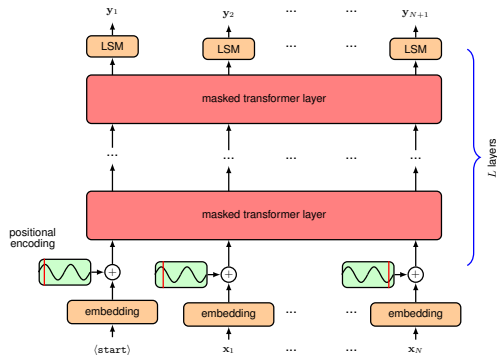
Philosophical interpretation: Deep learning is to give a strong enough model that can easily overfit any data. The task of learning is to learn the model parameters by **simultaneously** fitting the data and searching the **right complexity** (d_{VC}^*) of the model (by ways of avoiding overfitting).

Transformer and Large Language Models

The key part of a Transformer is the **attention**:

$$\mathbf{Y} = \text{attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

We often use **multi-head attention**.



Transformer and Large Language Models

- ▶ LLMs are mainly based on transformer. It has pre-training and post-training stages.
- ▶ The transformer/LLM essentially solves a **multi-class logistic regression** problem with learned features. For any input sequence, the next token acts as the groundtruth label for classification (**Self-Labeling / Next-Token Prediction**).
- ▶ Given training data $\mathcal{D} = \{x_i\}$ where each x_i is a sequence, the learning problem of LLMs can be formulated as minimizing the negative log-likelihood of the generation probability:

$$\hat{\theta} \leftarrow \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\theta) = \sum_{i \in \mathcal{D}} \sum_{j=1}^m -\log(\mathbb{P}_{\theta}[x_{i,j} | x_{i,1:j-1}]) . \quad (2)$$

- ▶ **Reasoning Models:** RLVR (GRPO) uses self-generated data for improving its math ability.

▪

The End.

Special thanks to

TAs and Your Participation ;-)